Cryptography and Network Security | Spring 2026
CompScan: Identifying Vulnerable Execution Paths in APK Exported Components

Final Project Proposal – Group 7

林紘騰 (B13902083)
National Taiwan University
Taipei, Taiwan

周世恩 (R14944076)
National Taiwan University
Taipei, Taiwan

曾家振 (B13705039)
National Taiwan University
Taipei, Taiwan

方婷 (B13705032)
National Taiwan University
Taipei, Taiwan

“‘latex

1 Abstract

Static analysis is widely used to identify security weaknesses in Android applications before deployment. However, existing tools often report a large number of findings without clearly indicating whether the reported issue is reachable through an externally exposed execution path. In this project, we present CompScan, a lightweight static APK scanner designed to identify potentially vulnerable execution paths in Android applications. CompScan focuses on exported components, deep links, inter-process communication surfaces, WebView-related risks, and insecure network configurations.

We evaluate CompScan on selected test cases from the OWApp Benchmarking Suite, focusing on the Platform and Network categories. For comparison, we use MobSF as a baseline scanner and normalize its reports into the same schema as CompScan. We manually construct a ground-truth dataset from the benchmark documentation and compare scanner findings against it using precision, recall, and F1 score. Our preliminary results show that CompScan produces substantially fewer false positives than MobSF under our evaluation framework, especially in the Network and Platform categories. Nevertheless, the relatively low recall indicates that CompScan still requires broader rule coverage and deeper code-level analysis.

2 Introduction

Android applications expose multiple entry points to the operating system and to other applications, including activities, services, broadcast receivers, content providers, and deep links. If these entry points are improperly protected, they may allow attackers to trigger sensitive functionality, access private data, or exploit insecure network behavior. Therefore, identifying vulnerable execution paths from externally reachable components is an important task in Android application security analysis.

This project aims to develop a static APK scanner, named CompScan, for detecting security-relevant execution paths in Android applications. Unlike general-purpose vulnerability scanners that may report a large number of isolated issues, CompScan emphasizes whether a potentially vulnerable behavior is associated with an exposed component or an externally triggerable interface. The scanner analyzes manifest-level information, exported components, deep link declarations, selected code-level patterns, and insecure network configurations. It then generates structured reports in JSON and Markdown formats, and attempts to provide proof-of-concept references when sufficient information is available.

To evaluate the scanner, we use the OWApp Benchmarking Suite as the benchmark source and compare CompScan with MobSF, a widely used mobile security analysis framework. We focus on the Platform and Network categories because they are closely related to Android component exposure, inter-process communication, WebView behavior, and insecure transport-layer configurations. The goal of the evaluation is not only to measure detection accuracy, but also to analyze the trade-off between strict path-based filtering and vulnerability coverage.

3 Related Work

The OWApp Benchmarking Suite provides a collection of Android security test cases aligned with common mobile security weaknesses. The benchmark contains multiple vulnerability categories, including Authentication, Code, Crypto, ListedAPKs, Network, ObfuscApks, Platform, Resilience, and Storage. Each test case contains a vulnerable APK and a corresponding Markdown description that explains the intended vulnerability and the implementation details of the application.

In this project, we select test cases from the Platform and Network categories. The Platform category includes issues related to Android components, exported interfaces, inter-process communication, and platform-level misuse. The Network category includes insecure communication behaviors such as cleartext traffic, improper TLS configuration, WebView-related network risks, and certificate validation problems. These categories are well aligned with our goal of identifying vulnerable execution paths that may be triggered through Android application interfaces.

We use MobSF as a baseline tool for comparison. MobSF provides comprehensive static and dynamic analysis capabilities for Android applications, but its raw reports may contain findings that are difficult to compare directly with our scanner output. Therefore, we normalize MobSF reports into a unified intermediate format before

evaluation. This allows both tools to be compared using the same ground truth and the same matching procedure.

4 Methodology

4.1 System Overview

The overall evaluation pipeline consists of four major stages: benchmark preparation, scanner execution, report normalization, and metric-based evaluation.

First, each APK from the selected benchmark categories is analyzed by CompScan. The scanner extracts manifest-level metadata, identifies exported components, detects deep link declarations, checks selected network-related configurations, and applies rule-based vulnerability patterns. The output is a structured report containing the detected findings, their associated components or paths, severity and confidence information when available, and optional proof-of-concept references.

Second, the same APKs are analyzed using MobSF. Since MobSF produces reports in a different format and with a different terminology, we normalize its findings into the same schema used by CompScan. This normalization step makes it possible to compare findings across tools without relying on tool-specific report structures.

Third, we manually inspect the Markdown documentation provided by the OWApp benchmark. For each test case, we extract the intended vulnerability and encode it into a JSON-based ground-truth file. This ground truth is used as the reference for determining whether a scanner finding should be counted as a true positive, false positive, or false negative.

Finally, the evaluation module compares the findings produced by CompScan and MobSF against the ground truth. The comparison uses category matching, keyword matching, and path-based matching to reduce the effect of different naming conventions across tools. Based on the matching result, we compute precision, recall, and F1 score for each scanner and each benchmark category.

4.2 Components

CompScan consists of several components.

- **Manifest Parser.** This component extracts application metadata from the APK, including package name, SDK versions, permissions, exported components, component types, intent filters, authorities, and relevant security attributes.
- **Deep Link Analyzer.** This component identifies intent filters that define externally reachable URI schemes, hosts, and paths. Such deep links may serve as entry points for triggering sensitive application behavior.
- **Risk Rule Engine.** This component applies rule-based checks to identify potentially risky configurations, such as exported components without sufficient protection, exposed content providers, cleartext traffic settings, and insecure network-related declarations.
- **Vulnerability Pattern Matcher.** This component searches for selected code-level or configuration-level signals associated with known Android security weaknesses. These signals are used to support the classification of findings.
- **PoC Generator.** When sufficient information is available, CompScan attempts to generate proof-of-concept references, such as commands or scripts that demonstrate how an exposed component or deep link may be triggered.
- **Report Generator.** The scanner generates both machine-readable JSON reports and human-readable Markdown reports. These reports summarize the detected findings, affected components, risk tags, confidence levels, and supporting evidence.

4.3 Report Format

The report generated by CompScan is designed to support both manual inspection and automated evaluation. Each finding contains the vulnerability category, affected component or path, supporting evidence, confidence level, severity or priority when available, and explanatory notes. The JSON report is used by the evaluation script, while the Markdown report is intended for human review. In addition, the scanner may generate a proof-of-concept reference when the vulnerable entry point can be represented concretely, such as an exported activity, content provider URI, or deep link.

To compare CompScan with MobSF, MobSF reports are normalized into the same intermediate format. This normalization reduces differences caused by report structure, naming conventions, and tool-specific terminology. As a result, the evaluation focuses on whether the underlying vulnerability is correctly identified rather than whether two tools use the same wording.

5 Evaluation

5.1 Evaluation Design

During evaluation, we compare the vulnerable execution paths reported by CompScan and MobSF against the manually constructed ground truth. For each benchmark case, the ground truth specifies the expected vulnerabilities based on the benchmark documentation. A scanner finding is counted as a true positive if it matches an expected vulnerability with sufficient semantic and path-level similarity. Findings that do not match any ground-truth item are counted as false positives, while ground-truth items not detected by the scanner are counted as false negatives.

Before matching, we filter findings that are likely to originate from third-party libraries or unrelated framework code. This step is especially important for MobSF because its raw reports may include a large number of generic findings that are not directly related to the benchmark vulnerability. After filtering, we perform category matching, keyword matching, and path matching. Each matching dimension contributes to a score, and the final score determines whether a finding should be considered a match.

We compute the following metrics:

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

We also record high-confidence and high-priority findings to analyze whether scanner-provided confidence or severity information can improve result ranking or threshold-based filtering. Since the current ground truth does not provide continuous numerical severity or confidence labels, severity-score MAE and confidence-score MAE are not reported.

5.2 Network Category Results

In the Network category, CompScan achieves higher precision and recall than MobSF under our current evaluation framework. MobSF reports 37 scoped findings, but none of them are matched to the ground-truth vulnerabilities after normalization and matching. In contrast, CompScan reports only 8 scoped findings, among which 7 are true positives. This leads to a precision of 0.8750 and a recall of 0.4375.

The result suggests that CompScan is more effective at filtering out unrelated network findings in this benchmark setting. This is likely because CompScan focuses on vulnerability patterns that are explicitly associated with the selected benchmark cases and attempts to preserve the relationship between the issue and the relevant execution path. However, the recall of 0.4375 also indicates that CompScan still misses more than half of the expected Network-category vulnerabilities. Therefore, additional rules are needed to cover a broader range of network misconfigurations, TLS-related issues, and WebView-specific behaviors.

5.3 Platform Category Results

In the Platform category, CompScan again achieves substantially higher precision than MobSF. MobSF produces 73 scoped findings, but only 3 are matched to the ground truth, resulting in a precision of 0.0411. CompScan produces 6 scoped findings, of which 5 are true positives, resulting in a precision of 0.8333. CompScan also achieves a higher recall than MobSF in this category, although the absolute recall remains low at 0.2000.

These results show that CompScan is more conservative than MobSF. It reports fewer findings, but a much larger proportion of its findings are relevant to the benchmark vulnerabilities. This behavior is consistent with the design goal of CompScan, which prioritizes externally reachable or security-relevant execution paths rather than reporting all potentially suspicious patterns. However, the low recall also shows the limitation of the current rule set. Some Platform-category vulnerabilities require deeper data-flow analysis, more complete inter-component communication modeling, or more precise recognition of Android framework behavior.

5.4 Overall Analysis

Across the selected Network and Platform benchmark cases, CompScan demonstrates a high-precision but limited-recall behavior. This trade-off is expected because the scanner applies stricter filtering and focuses on findings that can be associated with concrete components or execution paths. Compared with MobSF, CompScan produces fewer findings and significantly reduces false positives. This makes the generated reports easier to inspect manually and more suitable for vulnerability triage.

However, the current implementation is not yet comprehensive. The scanner may miss vulnerabilities whose evidence is not directly visible from the manifest or simple static patterns. For example, vulnerabilities involving complex control flow, runtime-generated URLs, dynamically registered components, or library-mediated network behavior may not be detected reliably. Therefore, future improvements should include broader vulnerability rules, more precise code-level analysis, and better modeling of Android inter-component communication.

6 Discussion

The evaluation highlights the main advantage and limitation of CompScan. Its main advantage is precision. By focusing on exported components, deep links, IPC surfaces, and selected network configurations, CompScan avoids many generic findings that are not directly related to the benchmark vulnerability. This makes its reports more concise and easier to interpret.

The main limitation is coverage. Although CompScan performs better than MobSF under our current matching framework, its recall remains low in both evaluated categories. This indicates that the current rule set captures only a subset of the vulnerabilities present in the benchmark. In particular, some vulnerabilities require deeper semantic understanding of the application code, such as tracing data from external inputs to sensitive sinks, detecting dynamically constructed network clients, or reasoning about custom certificate validation logic.

Another limitation comes from the evaluation process itself. Because different tools use different terminology and report formats, cross-tool comparison requires normalization and approximate matching. Although our evaluation uses category, keyword, and path matching to reduce this problem, some mismatches may still occur. Therefore, the reported numbers should be interpreted as results under our current normalization and matching design rather than as an absolute measurement of each tool's full capability.

Overall, CompScan is useful as a focused static analysis tool for identifying high-confidence vulnerable execution paths in Android APKs. It is especially suitable for cases where concise reports and low false-positive rates are important. Future work should focus on improving recall while preserving the current precision advantage.

7 LINE RCE

This section is reserved for a case study on the LINE remote code execution issue. The final version should describe the threat model, vulnerable entry point, triggering condition, exploitation path, and the relevance of the case study to the design of CompScan. Since the current draft does not yet include the technical details of this case study, we leave this section as future content.

8 Conclusion

In this project, we developed CompScan, a static APK scanner for identifying vulnerable execution paths in Android applications. The scanner focuses on exported components, deep links, IPC-related surfaces, WebView-related risks, and insecure network configurations. It generates structured JSON and Markdown reports and can provide proof-of-concept references when sufficient information is available.

We evaluated CompScan on selected Platform and Network test cases from the OWApp Benchmarking Suite and compared it with

Table 1: Evaluation results on the Network category.

Tool	Cases	Expected	Raw	Scoped	TP	FP	FN	Precision	Recall	F1
MobSF	5	16	42	37	0	37	16	0.0000	0.0000	N/A
CompScan	5	16	9	8	7	1	9	0.8750	0.4375	0.5833

Table 2: Evaluation results on the Platform category.

Tool	Cases	Expected	Raw	Scoped	TP	FP	FN	Precision	Recall	F1
MobSF	10	25	85	73	3	70	22	0.0411	0.1200	0.0612
CompScan	10	25	7	6	5	1	20	0.8333	0.2000	0.3226

MobSF. The results show that CompScan achieves substantially higher precision than MobSF under our evaluation framework, mainly because it applies stricter filtering and focuses on findings associated with relevant execution paths. However, its recall remains limited, indicating that the scanner requires broader rule coverage and deeper static analysis to detect more vulnerability types.

These results suggest that path-aware static analysis can reduce false positives in Android security scanning, but achieving high recall requires more complete modeling of Android application behavior. Future work will extend CompScan with additional vulnerability patterns, improved code-level analysis, and more precise inter-component communication tracking. “‘